Finding your browser's developer tools

All modern browsers contain a suite of developer tools. While all developer tools perform the same functions, they can be found in different places depending on which browser you use. Here is a breakdown of how to find the developer tools in each browser.

- *Chrome*—Choose the Chrome menu at the top-right of your browser window, then choose Tools > Developer Tools. Or press F12 or Ctrl + Shift + I (or Cmd + Opt + I on a Mac). For more information, see https://developers.google.com/chrome-developer-tools/.
- *Firefox*—Choose the Tools menu, then choose Web Developer > Toggle Tools. Or press F12 or Ctrl + Shift + I (or Cmd + Opt + I on a Mac). For more information, see https://developer.mozilla.org/en-US/docs/Tools.
- *Internet Explorer*—Internet Explorer's developer suite is called F12 because you can launch the developer tools by pressing F12. For more information, see http://msdn.microsoft.com/library/ie/bg182326(v=vs.85).
- *Opera*—Opera's developer suite is called Dragonfly and it can be found by choosing the Tools menu, then Advanced > Opera Dragonfly. Or press Ctrl + Shift + I (Cmd + Opt + I on a Mac). For more information, see http://www.opera.com/dragonfly/.
- *Safari*—To use Safari's developer tools, you must first enable the feature by clicking Show Develop Menu in the Menu Bar found in Safari's preferences under the Advanced pane. This will show a Develop menu in the toolbar. Next, choose Develop > Show Web Inspector. Or press Cmd + Opt + I after enabling the Develop menu. For more information, see https://developer.apple.com/safari/tools/.

The developer tools contain many different features, including:

- *Console*—Logs errors and other useful messages, and enables developers to interactively execute JavaScript on a page.
- *JavaScript debugger*—Allows the developer to set breakpoints and trace through JavaScript code.
- *Network monitor*—Logs the request and responses for all network traffic. Can be filtered by resource type.
- *DOM inspector*—Enables the developer to view the DOM associated with elements on the page.
- *Profiler*—Tracks the performance of actions on the site, and enables the developer to uncover performance bottlenecks.

When debugging CORS requests, we'll focus on two features in particular: the console and the network monitor. The examples from the rest of this chapter will use Chrome, but the developer tools from any browser can perform the same functions. Let's see how you can use these tools to debug CORS requests.